

The Logo Detection in an Illegitimate Web Page

Kanokporn Pakhamwang¹, Narathip Sriherun¹, Noppason Waikawee¹, Promchat Jaijith¹, Suppakarn Chansareewittaya^{1,2}, and Mahamah Sebakor^{1,2,*}

¹School of Information Technology, Mae Fah Luang University

²Computer and Communication Engineering for Capacity Building Research Center, Mae Fah Luang University

*Corresponding author: mahamah.seb@mfu.ac.th

Abstract—The growth of the online market is rapid. On the other hand, the fraud of brand products are significant serious issue. To mitigate these issues we proposed an intelligent system called The logo detection on an illegitimate web page. we use Convolutional Neural Networks (CNN) for detecting online fake logos on unauthorized web pages. moreover, the evaluation of our performance is presented. Finally, we apply our method to the real world through an online search of the Mae Fah Luang University's logo on the illegitimate web page. The result outstanding shows us that our proposed is useful for the fake logos detection.

Keyword: machine learning, WHO IS, domain, dark website, smart detection

I. INTRODUCTION

The logo we see today We cannot know if a logo is copyrighted or authorized to be used or traded on the dark web. So we need AI to detect abuse. Then scan for websites that use that logo to verify that it is real or not by using image processing to verify images or logos on that website.

In recent years, the detection and recognition of logos in images have gained significant attention due to their various applications in areas such as brand monitoring, image retrieval, and advertising analysis. Convolutional Neural Networks (CNNs) have emerged as a powerful tool for logo detection, allowing automated and accurate identification of logos within images.

The objective of this project is to explore the capabilities of CNNs in detecting and localizing logos in images. By leveraging the hierarchical and adaptive nature of CNN architectures, we aim to develop a robust and efficient logo detection system. This system will be able to automatically identify the presence and location of logos, even in complex and cluttered scenes [1]

The project involves several key steps. Initially, we will gather a diverse dataset of images containing different logos from various domains. The dataset will serve as the foundation for training and evaluating our CNN-based logo detection model. Preprocessing techniques, such as data augmentation and normalization, will be applied to enhance the model's performance and generalization capabilities.

Next, we will design and implement a CNN architecture suitable for image prediction. The architecture will consist of convolutional layers for feature extraction, pooling layers for spatial down-sampling, and fully connected layers for classification. The model will be trained using optimization algorithms such as stochastic gradient descent, which iteratively adjusts the model's weights to minimize the prediction error. During the training process, the CNN will learn to recognize and extract discriminative features from the images. It will develop an understanding of the underlying patterns and characteristics that differentiate one category from another. The model's performance will be continuously monitored and evaluated using validation techniques, such as cross-validation or a separate validation dataset.

After training, we will evaluate the CNN model's performance on a test dataset that consists of unseen images. Metrics such as accuracy and precision, will be computed to assess the model's predictive abilities. We may also employ techniques like confusion matrix analysis to gain insights into the model's strengths and weaknesses for different classes.

The successful completion of this project will result in a trained CNN model capable of accurately predicting the content or category of new images. The developed system can have practical applications in various domains, including image recognition, object detection, and content-based recommendation systems. Additionally, the project will contribute to the growing body of knowledge in the field of deep learning and image analysis.

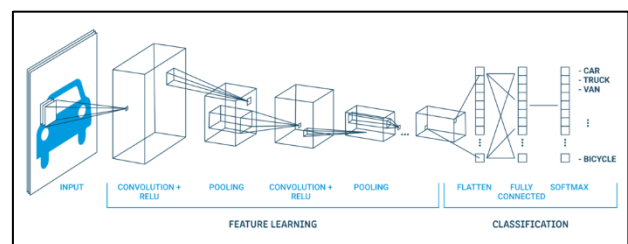


Fig. 1. An Example Convolutional Neural Networks (CNNs).

II. RELATED WORK

The paper proposes a framework for logo detection using deep region-based convolutional networks. with Flickr Logo-32 Datasets, Faster R-CNN frameworks with several widely used classification models for logo detection. To enable our design to become successful we will start by describing the related work on Logo Detection [1] This document represents The challenge with logo detection is the lack of a public logo dataset and the design of an efficient logo detection structure. They have Constructing Faster R-CNN frameworks with several widely used classification models for logo detection. Transfer learning methods are introduced in the training process , Clustering is used to guarantee proper hyper-parameters and more precise anchor points of the RPN. [2] They represent a method for automatically collecting representative logo images from the internet without any human labeling or seed images. The proposed method uses Multiple Dictionary Invariant Sparse Coding (MDISC) algorithm to collect high-quality logo images. [3] In-depth analysis and discussion of existing works in many aspects of logo detection, including data sets, pipelines employed, task kinds, detection algorithms, loss functions, and their contributions and limits.[4] This paper summarizes the current image processing technology, compares various methods, and explains their limitations. It introduces machine learning algorithms in image processing technology, applies convolution neural network to feature extraction in image processing, and proves the superiority of image processing technology based on machine learning by effectively improving the accuracy of image segmentation, image classification, and target detection. [5] The paper proposes a face image recognition method based on Convolutional Neural Network (CNN).The paper introduces a multi-layer information fusion network that combines the activation value output by VGG16 structure and the activation value in the multi-layer information fusion network and unsupervised learning methods for correcting tagged image data in the database. [6] The contributions of this paper are learning framework for face recognition based on Convolutional Neural Network (CNN) and Kernel Extreme Learning Machine (KELM) called CNN-KELM. Utilizing CNN for extracting the deep features of input data, followed by KELM on face recognition system. Evaluating the presented technique on multi-class classification problem using different face databases.Verifying the enhanced performance of the presented technique when compared with various state-of-the-art approaches. [7] The contributions of this paper areThe development of a deep learning system for automatic logo detection in real-world images.The use of the Faster R-CNN framework as the basis for the logo detector and comparison of its performance with other models such as Mask R-CNN and RetinaNet.A systematic detection performance comparison of various models on multiple popular datasets including FlickrLogos-32, TopLogo-10, and QMUL-OpenLogo benchmark.

III. PROPOSED APPROACH

The template iterates over the contents of the `data_dir` directory, which contains subdirectories representing different image classes. It assigns each subdirectory name to the variable `image_class`.It then iterates over the files in the subdirectory corresponding to the current image class. The complete path of each image file is formed by combining the `data_dir` path, `image_class` subdirectory, and image file name, creating the variable `image_path`. A try-except block is used to handle any exceptions that may occur while processing the image file. Inside the try block, the image file is read using OpenCV's `imread()` function, and its file type is determined using the `imghdr.what()` function, which assigns the result to the variable `tip`.If the detected image type (`tip`) is not in the list of allowed image extensions (`image_exts`), it means the image has an invalid or unwanted format. In such cases, a message is printed indicating that the image is not in the expected extension list.

Optionally, if you uncomment the line `os.remove(image_path)`, it will remove the invalid image file from the disk. If any exception occurs during the image processing, the code jumps to the except block, which prints a message indicating that there was an issue with the current image file. This commented out, meaning it is not currently executed. If uncommented, it would remove the problematic image file from the disk. Overall, will read each image file, check its type, and remove any images with unsupported formats or handle exceptions that may occur during the process. It ensures that only valid and usable image files remain in the dataset. The provided code snippet utilizes matplotlib to create subplots and display a batch of images with their corresponding titles.

Creates a figure and axes objects for subplots using the `subplots()` function from matplotlib. It specifies that there will be four subplots arranged in a single row (`ncols=4`) and sets the size of the figure to (20, 20) inches (`figsize=(20, 20)`). The resulting figure object is assigned to the variable `fig`, and the array of axes objects is assigned to `ax`.

Initiates a loop that iterates over a batch of images. `batch` is assumed to be a collection that contains both the images and their corresponding labels. `batch[0]` represents the array of images, and `batch[1]` represents the array of labels. The `enumerate()` function is used to get both the index (`idx`) and the corresponding image (`img`) from the first four elements (`[0:4]`) of the batch.

Displays the image on the subplot at index `idx` (based on the iteration). `ax[idx]` refers to the specific subplot associated with the current index. The `imshow()` function is used to display the image, with `img.astype(int)`

converting the image data type to integer (assuming it was originally stored as a float).

Sets the title for the subplot at index `idx`. `ax[idx].title` accesses the title of the subplot, and `set_text()` is used to assign the corresponding label (`batch[1][idx]`) as the title. It assumes that the labels are stored in `batch[1]`. Overall, this code creates a figure with four subplots, iterates over a batch of images, displays each image on the corresponding subplot, and sets the title of each subplot as the associated label. This allows for visualizing a subset of images with their corresponding labels in a single figure. For normalize the input data (images) by dividing each pixel value by 255. Normalization is a common preprocessing step in deep learning to scale the input data to a range between 0 and 1, making it easier for the neural network to learn and converge during training. By dividing each pixel value by 255, all pixel values will be in the range $[0, 1]$.

The resulting data will be a new dataset with the input data normalized and the labels unchanged, ready for training a neural network. (1) This adds a convolutional layer with 32 filters, a filter/kernel size of 3x3, and a ReLU activation function. The input shape specifies the shape of the input images to the model, which is (256, 256, 3) for RGB images (height, width, channels). (2) adds a max pooling layer with a pool size of 2x2. Max pooling helps reduce the spatial dimensions of the feature maps. (3) convolutional and max pooling layers, gradually increasing the number of filters to 64 and 128, respectively. This pattern helps the model learn higher-level features from the input images. (4) adds a flatten layer, which flattens the 2D feature maps into a 1D vector. This prepares the data for the subsequent fully connected layers. (5) This layer with 256 units and a ReLU activation function. Dense layers connect every neuron from the previous layer to every neuron in the current layer. (6) This adds the final dense layer with 10 units, which corresponds to the number of output classes. The activation function is softmax, which converts the output values into probabilities, representing the predicted probabilities for each class. (7) The model follows a typical pattern for image classification tasks using CNNs. The convolutional layers capture local patterns and features from the input images, while the max pooling layers downsample the spatial dimensions. The flatten layer and subsequent dense layers learn higher-level representations and perform classification based on the learned features.

Training a neural network model using TensorFlow and Keras. It includes a TensorBoard callback for visualizing training and validation metrics and stores the training history.

TensorBoard is a powerful visualization tool from TensorFlow that allows you to monitor and visualize various aspects of model training, such as loss, accuracy, and more. The `logdir` specifies the directory where the TensorBoard logs will be stored.

IV. PERFORMANCE EVALUATION

In this section, the proposed work is evaluated in terms of simulation and results. This paper also compares with other designs. As mentioned, Docker is used for simulation and testing.

The provided snippet visualizes the training and validation loss over epochs during the training of a neural network model. It uses matplotlib to create, the resulting plot will have the training loss and validation loss values on the y-axis and the epochs on the x-axis. It provides a visual representation of how the training and validation loss evolve over the training process. By monitoring these curves, you can gain insights into the model's performance, overfitting, and generalization capabilities. Ideally, you would want the training loss to decrease while the validation loss decreases and then levels off or starts increasing (indicating overfitting if it keeps decreasing).

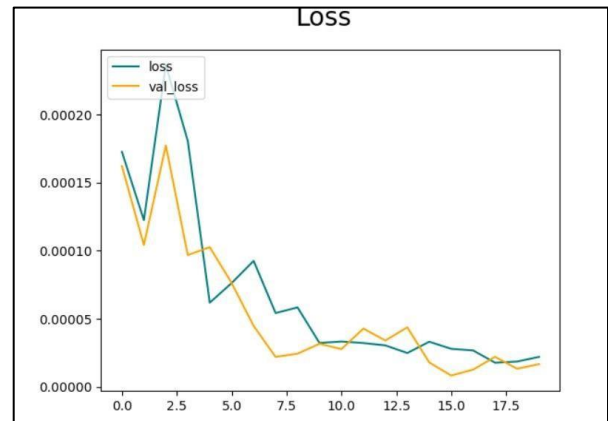


Fig. 2. Graph of loss.

Displays the plot with the training and validation accuracy lines and the legend. The resulting plot will have the training accuracy and validation accuracy values on the y-axis and the epochs on the x-axis. It provides a visual representation of how the training and validation accuracy evolve over the training process. By monitoring these curves, you can gain insights into the model's learning progress, overfitting, and generalization capabilities. Ideally, you would want both the training and validation accuracy to increase and converge as the training progresses, indicating that the model is learning and generalizing well to unseen data.

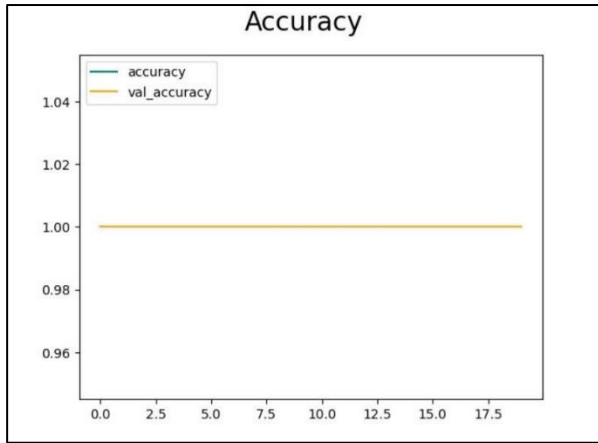


Fig. 3. Graph of accuracy.

V. EXPERIMENTAL RESULTS

In this section, the proposed logos are tested. in evaluating performance and results We used the logo of Mae Fah Luang University (MFU). The results show us that the logo test is realistic. But there is a limitation on image size.

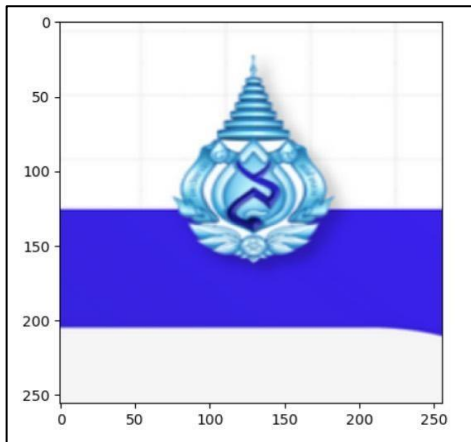


Fig. 4.1: Test logo Mae Fah Luang University (original).

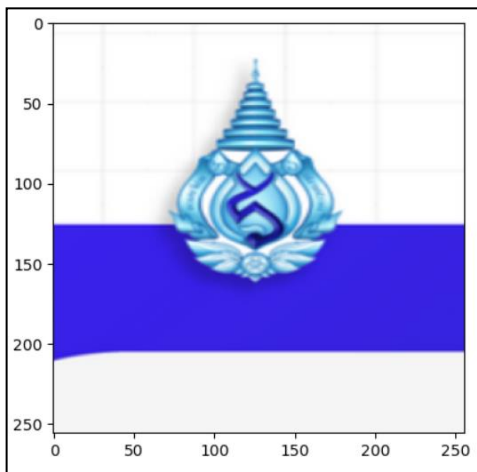


Fig. 4.2. Test logo Mae Fah Luang University (Flip horizontal).

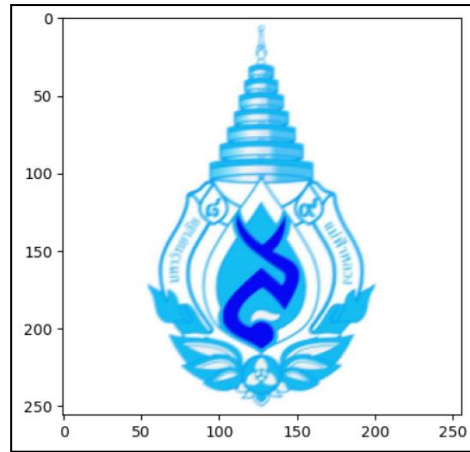


Fig. 4.3. Test logo Mae Fah Luang University (Blur 50%).

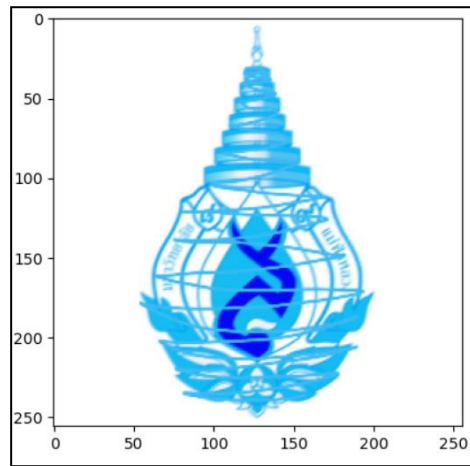


Fig. 4.4. Test logo Mae Fah Luang University (Cross logo).

VI. DISCUSSION and CONCLUSION

We have set the scope of Artificial Intelligence development from research planning and found the related research for reference from analysis design and system design attractive and usable and more efficient. We set the Artificial Intelligence development for a main roles:

- Develop AI System

This is new to us. We must start learning and starting over therefore taking longer than usual including study, research, data acquisition and experimentation and checking reference research details and verify the accuracy of the information. As a result, it was successful in developing AI, but the logo of the university could not be found. There is a limitation like image size. Can be further applied in business or in a way of fame that may damage reputation or business or resell the logo without permission which makes it illegal Able to scan logos on both surface web and dark web.

ACKNOWLEDGEMENT

This work is supported by Mae Fah Luang University.

REFERENCES

- [1] Y. Li, Q. Shi, J. Deng, and F. Su, "Graphic Logo Detection with Deep Region-based Convolutional Networks," in 2017 , pp. 1-2. Retrived 19 September 2023, [Online] from: <https://shorturl.asia/bdsJS>.
- [2] L. Xiaobing and Z. Bo, "Automatic Collecting Representative Logo Images from the Internet," in 2013 Tsinghua Science and Technology, Dec. 2013, pp. 1-2. Retrived 19 September 2023, [Online] from: <https://shorturl.asia/hC3el>.
- [3] J. Sanghvi, J. Rathod, S. Nemade, H. Pancha, and A. Pavate, "Logo Detection Using Machine Learning Algorithm : A Survey," in 2023 IEEE International Conference on Communication System, Computing and IT Applications , Oct. 2023, pp. 136-137. Retrived 19 September 2023, [Online] from: <https://shorturl.asia/ciTLY>.
- [4] Q. Qiao, "Image Processing Technology Based on Machine Learning," in 2022 School of Information and Communication Engineering, Communication University of China, pp.1. doi: 10.1109/MCE.2022.3150659.
- [5] G. Lou and H. Shi, "Face Image Recognition Based on Convolutional Neural Network," in 2020 ARTIFICIAL INTELLIGENCE (AI)-DRIVEN SPECTRUM MANAGEMENT , pp. 117-120. Retrived 19 September 2023, [Online] from: <https://shorturl.asia/piZkC>.
- [6] B. Ahuja and V. P. Vishwakarma, "Convolutional Neural Network and Kernel Extreme Learning Machine for Face Recognition," in 2022 8th International Conference on Signal Processing and Communication (ICSC), Jul. 2017, pp. 1. doi: 10.1109/ICSC56524.2022.10009623
- [7] K. Palecek, "Deep Learning for Logo Detection," in 2022 8th International Conference on Signal Processing and Communication (ICSC) |, in 2019 This work was supported by the Technology Agency of the Czech Republic, pp. 1. Retrived 19 September 2023, [Online] from: <https://shorturl.asia/k2Qci>.